

Department of Computer Science & Engineering

Operating Systems

UE24CS242B

OS Unit 4 Orange Problem

Name: DHRUV SWATANTRAMATH

SRN: PES1UG24CS155

Section: 4C

Environment: Ubuntu 22.04 / local repository completion

Implemented Files

- object.c: object_write and object_read implemented with hashing, sharding, integrity checking, and atomic writes.
- tree.c: tree_from_index implemented with recursive subtree construction and tree object writing.
- index.c: index_load, index_save, and index_add implemented for the text-based staging area.
- commit.c: commit_create implemented and commit parsing fixed for correct log output.
- report.md and screenshots/: updated with screenshot references and written answers.

Implementation Summary

- Phase 1 implements content-addressable object storage with hash verification. Phase 2 implements tree serialization and root tree construction from staged files. Phase 3 implements the text-based staging index with atomic saves. Phase 4 implements commit creation, reference updates, and log traversal.

Validation Summary

- ./test_objects passes for object storage behavior. ./test_tree passes for tree parsing and deterministic serialization. Manual pes init, pes add, pes status, pes commit, and pes log flows work. make test-integration completes successfully.

Screenshots

Phase 1

Screenshot 1A — ./test_objects output

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

This confirms successful blob storage, deduplication, and integrity checking in the object store implementation.

Screenshot 1B — find .pes/objects -type f

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

The object files are sharded by the first two hexadecimal characters of the hash, which matches the intended content-addressable storage layout.

Phase 2

Screenshot 2A — ./test_tree output

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

All Phase 2 tests passed.

This shows that tree serialization and parsing are consistent and that the serialized ordering is deterministic.

Screenshot 2B — raw tree object via xxd

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ xxd .pes/objects/d8/f28203071e10a3bde605928368bc5b5
1871287a867d8d10382967099dde814 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2068  tree 49.100644 h
00000010: 656c 6c6f 2e74 7874 0005 c8d7 77d0 be85  ello.txt....w...
00000020: 748b 6788 d1b5 caf7 d697 54e6 e680 60c2  t.g.....T...`
00000030: 65ca 21cb d6c0 5e5f 43          e.!...^C
```

The raw dump shows the tree header followed by serialized entries containing mode, filename, and binary object hashes.

Phase 3

Screenshot 3A — pes init -> pes add -> pes status

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked:  Makefile
untracked:  test_tree
untracked:  object.c
untracked:  tree.h
untracked:  test_tree.c
untracked:  test_sequence.sh
untracked:  commit.c
untracked:  tree.c
untracked:  pes.c
untracked:  commit.h
untracked:  pes.h
untracked:  index.c
untracked:  .gitignore
untracked:  test_objects
untracked:  test_objects.c
untracked:  .codex
untracked:  README.md
untracked:  index.h
```

The status output shows that staged files are tracked correctly and that no unstaged changes are reported immediately after adding them.

Screenshot 3B — cat .pes/index

```
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776662045 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776662045 6 file2.txt
```

The index stores the file mode, blob hash, modification time, size, and path in the expected text format.

Phase 4

Screenshot 4A — pes log

```

dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ ./pes log
commit c9ba218459250635381222529933fc30b828046ceb3446aa7e788cb61406aefc
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776662474

    Add farewell

commit 457ae3bb3c0642acbcc8d3bacdb2f410e25a8888e1ffbb8a5f5fe7d48ba1973a
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776662466

    Add world

commit 2abecb636c510f6d4ff88236d7f3890f6d2df541565fa0b618b235bb9cf32371
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776662457

Initial commit

```

The log confirms that commits are linked through parents and printed from newest to oldest with author, timestamp, and message metadata.

Screenshot 4B — `find .pes -type f | sort`

```

dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2a/becb636c510f6d4ff88236d7f3890f6d2df541565fa0b618b235bb9cf32371
.pes/objects/45/7ae3bb3c0642acbcc8d3bacdb2f410e25a8888e1ffbb8a5f5fe7d48ba1973a
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afd855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c9/ba218459250635381222529933fc30b828046ceb3446aa7e788cb61406aefc
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main

```

This listing shows the growth of repository metadata after multiple commits, including refs, the index, and newly created objects.

Screenshot 4C — `cat .pes/refs/heads/main` and `cat .pes/HEAD`

```

dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
c9ba218459250635381222529933fc30b828046ceb3446aa7e788cb61406aefc
ref: refs/heads/main

```

The output demonstrates the expected reference chain: HEAD points to the branch name, and the branch file points to the latest commit hash.

Final Integration Test

Integration Screenshot — `make test-integration`

```

dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:   file.txt
  staged:   hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: 843b640c24fe... Initial commit

Log after first commit:
commit 843b640c24fe0901a69e8b91ba79c4d6376b1df3d6a0e5b0068639e6b82ea91f
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776663082

Initial commit

```

```

--- Object Store ---
Objects created:
10
.pes/objects/bb/d69098bd9b9cc5934a610ab65da429b525361147Faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f887ec727b7ab7d4cf86f62f5126e2047cb4ad7886d2b30228e8e5e36c6ec8
.pes/objects/84/3b640c24fe0901a69e8b91ba79c4d6376b1df3d6a0e5b0068639e6b82ea91f
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0cc92
.pes/objects/b9/0467ee739ecff24c55eca6d634288b6c898ed4f2f5f5d3adae71b6fff9d68
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
dhruv@LAPTOP-SFEPND97:~/PES1UG24CS155-pes-vcs/PES1UG24CS155-pes-vcs$

--- Second Commit ---
Committed: b90467ee739e... Update file.txt

--- Third Commit ---
Committed: 6ef887ec727b... Add farewell

--- Full History ---
commit 6ef887ec727b7ab7d4cf86f62f5126e2047cb4ad7886d2b30228e8e5e36c6ec8
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776663082

    Add farewell

commit b90467ee739ecff24c55eca6d634288b6c898ed4f2f5f5d3adae71b6fff9d68
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776663082

    Update file.txt

commit 843b640c24fe0901a69e8b91ba79c4d6376b1df3d6a0e5b0068639e6b82ea91f
Author: DHRUV SWATANTRAMATH <PES1UG24CS155>
Date: 1776663082

    Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
6ef887ec727b7ab7d4cf86f62f5126e2047cb4ad7886d2b30228e8e5e36c6ec8

```

Together, these screenshots verify the full end-to-end workflow from repository initialization through staging, committing, history traversal, and reference updates.

Analysis Answers

Q5.1 Branching and Checkout

A branch in PES-VCS should be implemented as a file under `.pes/refs/heads/<branch>` that stores the target commit hash, exactly like the existing main branch file. A `pes checkout <branch>` operation would first verify that the branch file exists, then rewrite `.pes/HEAD` so that it contains `ref: refs/heads/<branch>`. After that, the working directory must be updated to match the tree pointed to by the branch's latest commit. That means reading the target commit object, reading its root tree, recursively materializing every file and directory into the working directory, and removing tracked files that exist in the current branch but not in the target branch. The complexity comes from the fact that checkout is not only a metadata change in `.pes/`; it also has to safely transform the user's actual filesystem state without overwriting uncommitted work or leaving the repository half-updated if an error occurs in the middle.

Q5.2 Dirty Working Directory Conflict Detection

The conflict check can be done by comparing three states: the HEAD snapshot, the index, and the working directory. For each tracked path in the current branch, PES-VCS should determine whether the working directory still matches the staged version recorded in the index. The fast path is to compare `stat()` metadata such as modification time and size against the index entry. If those values no longer match, the safe path is to re-read the file, hash it, and compare that hash with the blob hash stored in the index or current HEAD tree. If a file has local modifications and the target branch also wants a different version of that same path, checkout must refuse because switching branches would overwrite user data. In short, PES-VCS should reject checkout when a tracked file is dirty in the working tree and the target branch would update or delete that path.

Q5.3 Detached HEAD

In detached HEAD state, `.pes/HEAD` would contain a commit hash directly instead of a symbolic reference like `ref: refs/heads/main`. New commits made in this state still work: each new commit records the previous detached commit as its parent, so history continues normally, but no branch name moves forward to

remember those commits. As a result, the commits become hard to find later unless the user notes the hash manually. Recovery is straightforward if the user still knows one of the detached commit hashes: create a new branch file under `.pes/refs/heads/` pointing to that commit, or update an existing branch to that hash. Once a reference points to the detached history again, the commits are no longer orphaned and can be reached through normal log traversal.

Design assumption for Q5: these checkout answers assume PES-VCS updates only tracked files and rebuilds the working directory from tree objects, rather than trying to preserve arbitrary untracked files automatically.

Q6.1 Garbage Collection and Space Reclamation

Garbage collection should use a mark-and-sweep algorithm. The mark phase starts from every live reference, such as all files under `.pes/refs/heads/` and also `HEAD` if it stores a direct commit hash. For each referenced commit, PES-VCS recursively marks the commit itself, its parent commits, its root tree, every subtree reachable from that tree, and every blob reachable from those trees. A hash set is the right data structure for tracking reachability efficiently, because membership tests and inserts are close to $O(1)$. After marking, the sweep phase walks `.pes/objects/` and deletes any object whose hash is not in the reachable set. For a repository with 100,000 commits and 50 branches, the number of visited objects is on the order of the distinct reachable commits, trees, and blobs, not $100,000 * 50$, because many branches usually share large parts of history. In practice, GC would still need to visit a very large number of objects, likely hundreds of thousands or more in a moderately sized repo.

Q6.2 Why Concurrent GC Is Dangerous

Running garbage collection concurrently with commit creation creates a race between "object becomes reachable" and "reference is updated." During commit, PES-VCS first writes blobs, trees, and the new commit object, and only at the end updates the branch reference. If GC scans references in the middle of that sequence, it may not yet see the new commit from any branch, so it can conclude that the newly written tree or commit objects are unreachable and delete them. Immediately after that, the commit path might update `refs/heads/main` to point at a commit whose underlying objects were just removed, corrupting the repository. Real Git avoids this with coordination and conservative retention: reference updates are atomic, very new objects are protected, and GC is careful not to collect objects that may still become reachable moments later.

Failure consequence for Q6: the most visible symptom would be a branch ref that points to a commit hash which exists, but whose tree or blob dependencies have already been deleted, causing future reads or `log/tree` walks to fail.